Name: Shrisarth kutwad Class: SOC Batch: B
Subject: Python Roll No.: 69 Exp. No.: 2
Name of the Experiment:

Marks	Teacher's Signature with date
10	

Performed on:
Submitted on:

Q.1) List the python sequence data type and State one key characteristic of each with respect to mutability or ordering.

→ In python, Sequence data types store collections of items in a specific order.

1. List :- ordered sequence mutable (elements can be changed after creation)
2. Tuple :- ordered sequence Immutable (cannot be changed once created)
3. String :- ordered sequence of characters immutable (Characters can not be modified)
4. Range :- ordered sequence of numbers. Immutable (cannot modify values, only generate them).

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐
Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐
Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

Print("every Second Character:" every_second
Print("length of string:" Length)
Print("maximum_character:", max_char)

- indexing → Accessing single Characters (text[0], text[-1])
- Slicing → extracting part of the string [text[0:6], text[:10]]
- Built in functions used
  - len(); → finds leng
  - max() → Finds the highest Characters (based on Ascii value)

Q.4] Analyze the difference between equality (==) and identity when used with sequence types. Discuss how object mutability and memory reference affect program behaviour.

→ Difference between == and is in python

- The equality operator (==) Compares the values or Contents of two sequence objects.
- The identity operator (is) Compares the memory location of two objects.
- two sequnces may how the Same values but be Stored in different memory locations so == return true while is returns false
- For mutable sequence like lists, if

two variable refer to the same object, changes made through or variable affect the other.

- For immutable sequence like Strin and tuples, values can not be mo fied, any change creates a new object, reducing side effects.

Q.5 Evaluate, the suitability of lists, se sets and dictionaries for stories and managing unique student re rds. Justify which data structur is most efficient and why.

→ Suitability of lists, sets and dictio es for unique student records, t Choice of dates structure affects efficiency, dates integrity, and e of access.

1. Lists

- Lists store elements in ordered seq ence
- They allows duplicate entries, so extra logic is required to ensure uniquenti
- searching for a student record specialy for large data bases.

2. Sets
- Sets store only unique elements automaticaly.
- They are unordered and do not support indexing.
- Fast membership checking (o(1) average (time).

3. Dictionaries
- Dictionaries store data as key value pairs
- keys are unique, ensuring duplicate student IDs.
- very fast accross, insertion and deletion average time.
- Can Store Structed records as values.

Q.6 Design and implement a python program that appropriate sequence types Indexing arslicing Supit in functions to manage and process Student data efficiency.

→

```
Students = {
1: {"name": "Amit", "marks": [78,85,90]}
2: {"name": "Sneha", "marks": [88,91,81]}
}
for roll, data in Students.itemD():
    marks = data["marks"]
```

```
first_mark=marks[0]
top_two=sorted(marks,reverse:True
                                [:2]

total=Sum(markes)
average: total/len(marks)

Print(F"\n Roll NO:{roll}")
Print(F"name:{data['name']}")
Print(F'marks {marks}")
Print(F"Topfirst mark:{first_mark}")
Print(F:Top two marks:{top-two}")
 Print(F"Total:{total}, Average:{Aver
```

- Dictonary ensures unique Stude
  Stores marks
- Indering (marks[0] and Slicing[:2
  are used
- Build in functions: sum(), Len(), sor
  ()

Q.7 write a program to print the first and last Character of String)

→

```
text=input("Enter a String:")
Print("First Character:",text[0])
Print("Last Character:",text[-1])
```

- text[0] → accesses the first Charact
- text[-1] → access the last Charact
  using negative indeming

Q.8 write a program to create a list, tople, set and dictionary.

→
```
My_List=[10,20,30,40]

My_tople =(1,2,3,4)

My_set= {5,10,15}

My_dict={5,10,15}

My_dict = {
            "name":"Amit,"
            "age":20
            "branch" :"(st)"
  }
  ;
Print("List:" My_List)
Print("Tuple:" My_tuple)
Print("Set:", my_set)
Print("Dictionary:", my_dict)
```

Q.9 write a program to understand the the concept of idenfing.

→
```
text = "Python"
Print("String:" text)
Print("First Character:" , text[0])
Print("second Character:", text[1])
Print("Last Character:", text[-1])
```

```
numbers = [10, 20, 30, 40]
Print("In list:", numbers)
```

Q.19 Write a program to understand the concept of buit in function.

→

```
numbers = [10, 20, 30, 40, 50]

Print("List:", numbers)

Print("Length of List:", Len(numbe
Print("maximum value:", max(numbe
Print("minimum value:", min(numbe
Print("Sum of elements:", Sum(numbers
Print("Sorted list", sorted(numbers))
```